parameter_sensitivity_explorer

February 17, 2026

1 Parameter Sensitivity Explorer

This notebook performs a **vectorized grid search** using the ggTrader orchestrator api. It visualizes the profitability landscape to find robust parameter regions.

[5]:
```
import sys
import os
import pandas as pd
import numpy as np
import vectorbt as vbt
import plotly.graph_objects as go
from tabulate import tabulate

# Auto-reload custom modules
%load_ext autoreload
%autoreload 2

# Ensure project root is in path
project_root = os.path.abspath(os.path.join(os.getcwd(), '..',  'src'))
if project_root not in sys.path:
    sys.path.append(project_root)

from ggTrader.core.orchestrator import run_sensitivity_orchestrator

print("Environment initialized.")
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
Environment initialized.
```

[6]:
```
# --- Configuration ---
CONSTANTS = {
    "SYMBOLS": None,
    "SYMBOLS_FILE": os.path.join(os.getcwd(), "..", "data",␣
 ↪"top_10_USD_1095_movers.json"),
    "START_DATE": "2023-01-01",
    "END_DATE": "2024-12-31",
    "INTERVAL": "4h",
```

```
    "START_CASH": 10000,
    "PORTFOLIO_SHARE": 0.20,
    "FEES": 0.004,
}

print("Configuration loaded.")
```

```
Configuration loaded.
```

[7]:
```
# --- Define Parameter Grid ---
params = {
    "adx_threshold": list(range(15, 60, 5)),
    "adx_length": list(range(10, 50, 5)),
    "sar_acceleration": [0.02],  # does not vary much
    "sar_maximum": [0.2], # does not vary much
    "atr_multiplier": list(np.arange(1.0, 6.0, 0.5)),
    # "atr_multiplier": list(range(1, 10, 1)),
    "atr_length": list(range(10, 50, 5)),
    "use_dmp_cross": [True, False],
}

print("Parameter grid defined.")
```

```
Parameter grid defined.
```

[8]:
```
# --- Run Vectorized Analysis ---
# Note: show_progress=True enables VectorBT's tqdm progress bar
results = run_sensitivity_orchestrator(
    config=CONSTANTS, param_grid=params, save_results=False, show_progress=True
)

results_df = results["results_df"]
best_params = results["best_params"]

print("\nAnalysis Complete.")
```

```
Loading data…
Running Vectorized Sensitivity Analysis…
  0%|          | 0/11520 [00:00<?, ?it/s]


Analysis Complete.
```

[9]:
```
print("\nTOP 10 COMBINATIONS:")
print(tabulate(results_df.sort_values("Sharpe Ratio", ascending=False).
 ↪head(10), headers="keys", tablefmt="simple", showindex=False))

print("\nBEST PARAMETER SET:")
```

```
print(tabulate(pd.DataFrame([best_params]), headers="keys", tablefmt="simple",␣
↪showindex=False))
```

```
TOP 10 COMBINATIONS:
  adx_length    adx_threshold    sar_acceleration    sar_maximum  use_dmp_cross
atr_length    atr_multiplier    Sharpe Ratio
------------  ---------------  ------------------  -------------
---------------  ------------  ----------------  --------------
          40               55                0.02            0.2  False
40               2.5              inf
          40               55                0.02            0.2  False
30               5                inf
          40               55                0.02            0.2  False
40               5.5              inf
          40               55                0.02            0.2  False
35               3                inf
          40               55                0.02            0.2  False
35               3.5              inf
          40               55                0.02            0.2  False
35               2                inf
          40               55                0.02            0.2  False
35               2.5              inf
          40               55                0.02            0.2  False
25               3.5              inf
          40               55                0.02            0.2  False
25               4                inf
          40               55                0.02            0.2  False
30               1                inf

BEST PARAMETER SET:
  adx_length    adx_threshold    sar_acceleration    sar_maximum  use_dmp_cross
atr_length    atr_multiplier
------------  ---------------  ------------------  -------------
---------------  ------------  ----------------
          40               55                0.02            0.2  True
10                1
```

[10]:
```
# --- Visualization Helper ---
def show_heatmap(df, x_param, y_param, metric="Sharpe Ratio"):
    """Generates and displays a heatmap for the given parameter pair."""
    heatmap_data = df.pivot_table(
        index=y_param,
        columns=x_param,
        values=metric,
        aggfunc="mean"
    )
```

```
    fig = go.Figure(data=go.Heatmap(
        z=heatmap_data.values,
        x=heatmap_data.columns,
        y=heatmap_data.index,
        colorscale='Viridis',
        colorbar=dict(title=metric)
    ))

    fig.update_layout(
        title=f"{metric} Landscape: {y_param} vs {x_param}",
        xaxis_title=x_param,
        yaxis_title=y_param
    )

    fig.show()

print("Visualization helper defined.")
```

```
Visualization helper defined.
```

[11]:
```
# --- Generate Heatmaps ---
# You can list any pairs of parameters you want to explore
pairs_to_plot = [
    ("adx_threshold", "adx_length"),
    # ("sar_acceleration", "sar_maximum"),
    ("atr_multiplier", "atr_length"),
    ("adx_length", "atr_length"),
    ("use_dmp_cross", "adx_length"),
    ("use_dmp_cross", "adx_threshold")
]

for x, y in pairs_to_plot:
    show_heatmap(results_df, x, y)
```

[ ]:
```

```